Biểu diễn SAT tối ưu cho bài toán N-Queens: Đánh giá thực nghiệm các phương pháp mã hóa.

Trần Việt Toàn[a]

[a]Trường Đại học Công nghệ - ĐHQGHN

Tóm tắt nội dung

Bài báo này trình bày phương pháp giải bài toán N-Queens bằng cách chuyển đổi thành bài toán thỏa mãn logic mệnh đề (SAT) thông qua thư viện PySAT. Trọng tâm của nghiên cứu là thiết kế và đánh giá các kỹ thuật mã hóa (encoding) để biểu diễn ràng buộc "tối đa một" (At-Most-One - AMO) và "ít nhất một" (At-Least-One - ALO). Chúng tôi triển khai phương pháp mã hóa Sequential Counter và Commander nhằm giảm thiểu số lượng mệnh đề sinh ra so với phương pháp Pairwise truyền thống. Kết quả của cấu hình SAT sẽ được thiết kế để đối chiếu thực nghiệm với các bộ giải chính xác (ILP, CP) như Cplex, Gurobi và CP-SAT trên các ma trận kích thước lớn.

1. Giới thiệu

N-Queens là một bài toán tối ưu tổ hợp kinh điển, yêu cầu đặt N quân Hậu lên bàn cờ $N \times N$ sao cho không có bất kỳ hai quân Hậu nào tấn công nhau. Để giải quyết bài toán này trên quy mô lớn ($N = 50, 100, 500$), việc sử dụng các thuật toán quay lui (backtracking) thông thường là không khả thi. Thay vào đó, bài toán được dịch sang dạng chuẩn CNF (Conjunctive Normal Form) để giải bằng các công cụ SAT Solver hiện đại thông qua thư viện PySAT. Mục tiêu của báo cáo là so sánh hiệu năng của phương pháp SAT Solving với các phương pháp lập trình nguyên thủy (ILP) và lập trình ràng buộc (CP) thông qua các thông số cốt lõi: số lượng biến (n_v), số lượng mệnh đề (n_c), thời gian xây dựng mô hình và thời gian tìm kiếm.

2. Phương pháp biểu diễn (SAT Encoding)

2.1. Biến quyết định và Dạng chuẩn CNF

Gọi $X_{i,j}$ là biến Boolean đại diện cho ô ở hàng i, cột j trên bàn cờ kích thước $N \times N$.

$$X_{i,j} = \begin{cases} 1, & \text{nếu có quân Hậu đặt tại ô } (i,j) \\ 0, & \text{nếu ngược lại} \end{cases} \tag{1}$$

Bài toán N-Queens được cấu thành từ hai loại ràng buộc cơ bản: ALO (At-Least-One - Ít nhất một) và AMO (At-Most-One - Tối đa một).

2.2. Ràng buộc ALO (At-Least-One)

Ràng buộc ALO yêu cầu mỗi hàng và mỗi cột phải có ít nhất một quân Hậu. Mã hóa cho một hàng i có chiều dài N được biểu diễn trực tiếp bằng một mệnh đề tuyển (clause) duy nhất:

$$\bigvee_{j=1}^{N} X_{i,j} \tag{2}$$

orcid(s):

2.3. Tối ưu Ràng buộc AMO bằng Sequential Counter

Ràng buộc AMO cấm việc đặt nhiều hơn một quân Hậu trên cùng một hàng, cột hoặc đường chéo. Nếu sử dụng phương pháp Binomial (Pairwise) cơ bản, việc bắt cặp tất cả các biến sẽ sinh ra $\mathcal{O}(N^2)$ mệnh đề, gây bùng nổ bộ nhớ khi N lớn.

Để tối ưu, chúng tôi cài đặt phương pháp Sequential Counter (Bộ đếm tuần tự). Phương pháp này giới thiệu $N-1$ biến phụ (auxiliary variables) $S_1, S_2, \dots, S_{N-1}$ cho mỗi dãy N ô. Biến S_i mang ý nghĩa trạng thái: $S_i = 1$ (True) khi và chỉ khi tồn tại ít nhất một biến mang giá trị 1 trong tập tiền tố $(X_1, X_2, \dots, X_i)$.

Hệ thống luật của Sequential Counter được dịch sang CNF như sau:

- Luật biên trái: Nếu ô đầu tiên có Hậu, tín hiệu truyền đi được kích hoạt.

$$X_1 \to S_1 \equiv \neg X_1 \vee S_1 \tag{3}$$

- Luật truyền tin (với $1 < i < N$): Trạng thái được cập nhật và truyền tiếp dọc theo chuỗi.

$$(X_i \vee S_{i-1}) \to S_i \equiv (\neg X_i \vee S_i) \wedge (\neg S_{i-1} \vee S_i) \tag{4}$$

- Luật cấm AMO (với $1 < i < N$): Nếu tín hiệu từ tiền tố báo đã có Hậu, cấm đặt Hậu ở ô hiện tại.

$$S_{i-1} \to \neg X_i \equiv \neg S_{i-1} \vee \neg X_i \tag{5}$$

- Luật biên phải: Áp dụng lệnh cấm cho ô cuối cùng.

$$S_{N-1} \to \neg X_N \equiv \neg S_{N-1} \vee \neg X_N \tag{6}$$

2.4. Tối ưu Ràng buộc AMO bằng Commander Encoding

Khi quy mô bàn cờ tăng lên, phương pháp Sequential có thể tạo ra chuỗi lan truyền dài. Để kiểm soát cấu trúc biến phụ, chúng tôi áp dụng phương pháp Commander Encoding (Mã hóa Chỉ huy) . Kỹ thuật này chia các biến thành nhóm, tạo một biến đại diện cho mỗi nhóm rồi áp đặt AMO lên các đại diện đó; phương pháp này giúp tạo tính cục bộ (locality) theo khối và cực kỳ phù hợp cho các miền biến vừa và lớn [1]. Cho tập hợp n biến, thuật toán phân hoạch chúng thành các nhóm $G_1, G_2, \dots, G_m$ có kích thước cố định (ví dụ $g = 3$). Mỗi nhóm G_i được gán một biến chỉ huy (commander variable) C_i. Hệ thống luật được quy đổi sang CNF bao gồm:

- Luật Chỉ huy Sai: Nếu biến chỉ huy mang giá trị 0 (False), tất cả biến trong nhóm phải mang giá trị 0.

$$C_i \vee \neg X_k \quad (\forall X_k \in G_i) \tag{7}$$

- Luật Chỉ huy Đúng: Nếu biến chỉ huy mang giá trị 1 (True), có chính xác một biến trong nhóm mang giá trị 1 . Điều kiện này bao gồm ALO cục bộ:

$$\neg C_i \vee \bigvee_{X_k \in G_i} X_k \tag{8}$$

  và AMO nội bộ bằng cách áp dụng phương pháp Pairwise cho các cặp biến (X_a, X_b) bên trong G_i: $\neg X_a \vee \neg X_b$.

- Ràng buộc trên Chỉ huy: Bản thân các biến chỉ huy $C_1, \dots, C_m$ tiếp tục phải tuân thủ điều kiện AMO. Nếu số lượng chỉ huy lớn, quá trình gom nhóm được áp dụng đệ quy.

2.5. Tối ưu Ràng buộc AMO bằng Product Encoding

Bên cạnh cấu trúc lan truyền tuần tự và chia nhóm cục bộ, phương pháp Product Encoding (Mã hóa Tích) xử lý ràng buộc AMO bằng cách phân rã bài toán lớn thành các ràng buộc nhỏ hơn dựa trên cấu trúc lưới hai chiều [2]. Việc đưa cấu trúc mảng 1D về dạng lưới 2D giúp khống chế số lượng mệnh đề cấm một cách hiệu quả, đặc biệt tối ưu cho các không gian trạng thái lớn [3]. Quá trình mã hóa cho tập hợp n biến gốc $X_1, X_2, \dots, X_n$ được thực hiện qua các bước sau:

- Khởi tạo tọa độ ảo: Thiết lập một ma trận kích thước $p \times q$, với $p = \lceil \sqrt{n} \rceil$ và $q = \lceil n/p \rceil$ [2].
- Sinh biến phụ: Cấp phát p biến phụ $r_1, \dots, r_p$ đại diện cho các hàng ảo và q biến phụ $c_1, \dots, c_q$ đại diện cho các cột ảo. Lượng biến phụ được khống chế ở mức $\mathcal{O}(\sqrt{n})$ [3].
- Ánh xạ biến gốc: Mỗi biến X_k được ánh xạ tới một tọa độ duy nhất (r_u, c_v) [2] . Quy tắc kéo theo yêu cầu nếu X_k nhận giá trị 1 (True), thì hàng và cột tương ứng chứa nó cũng phải nhận giá trị 1:

$$X_k \to (r_u \wedge c_v) \equiv (\neg X_k \vee r_u) \wedge (\neg X_k \vee c_v) \tag{9}$$

- Ràng buộc trên trục: Ràng buộc AMO bằng kỹ thuật Pairwise truyền thống nay được áp đặt độc lập lên tập biến hàng $\{r_1, \dots, r_p\}$ và tập biến cột $\{c_1, \dots, c_q\}$. Bằng cách ép tối đa một hàng và một cột được phép mang giá trị 1 , tổng số lượng mệnh đề cấm sinh ra được giảm mạnh xuống mức $\mathcal{O}(n)$ [3].

2.6. Tối ưu Ràng buộc AMO bằng Binary Encoding

Khác với các cấu trúc lưới và mảng, Binary Encoding (Mã hóa Nhị phân) khai thác sức mạnh của hệ đếm cơ số 2 nhằm tối thiểu hóa tuyệt đối lượng biến phụ.

Cho tập hợp n biến gốc $X_1, X_2, \dots, X_n$, thuật toán xử lý theo các bước:

- Cấp phát biến phụ: Thuật toán xác định $k = \lceil \log_2 n \rceil$ và khởi tạo k biến phụ $Y_1, Y_2, \dots, Y_k$[4]. Lượng biến phụ được nén xuống mức $\mathcal{O}(\log_2 n)$[4].
- Định danh nhị phân: Mỗi biến X_i được gán cố định với một chuỗi nhị phân độ dài k [4].
- Luật ánh xạ trạng thái: Quy tắc kéo theo yêu cầu rằng, nếu X_i mang giá trị True, tập hợp các biến phụ Y bắt buộc phải thiết lập trạng thái chính xác theo cấu hình bit của X_i[4]. Công thức được biểu diễn dưới dạng:

$$X_i \to \bigwedge_{j=1}^{k} \phi(i, j) \tag{10}$$

  Trong đó, $\phi(i, j) = Y_j$ nếu bit thứ j trong chuỗi nhị phân của X_i là 1, và $\phi(i, j) = \neg Y_j$ nếu bit đó là 0[4]. Chuyển đổi sang chuẩn CNF, ta thu được các mệnh đề dạng $(\neg X_i \vee \phi(i, j))$.

Vì tập biến phụ $Y_1, \dots, Y_k$ chỉ có thể tồn tại ở một trạng thái nhị phân duy nhất, luật ánh xạ này tự động ngăn cản việc có nhiều hơn một biến X_i mang giá trị True. Tổng số lượng mệnh đề cấm sinh ra từ phương pháp này đạt mức tuyến tính logarit $\mathcal{O}(n \log_2 n)$[4].

3. Triển khai Thực nghiệm và Đánh giá (Benchmark)

3.1. Môi trường và Phương pháp đo lường

Thực nghiệm được lập trình bằng ngôn ngữ Python, tích hợp bộ giải Glucose3 từ thư viện PySAT. Để đánh giá khả năng mở rộng (scalability), chúng tôi tiến hành chạy thuật toán trên các ma trận $N \times N$ với kích thước lớn dần: $N = 50, 100, 500$[2].

Theo đúng chuẩn thiết kế thực nghiệm, chúng tôi cài đặt thêm các thư viện giải chính xác (exact solvers) để đối chứng trực tiếp với phương pháp SAT Solving, bao gồm Gurobi (đại diện cho MIP) và CP-SAT (đại diện cho Lập trình ràng buộc)[2] . Ma trận thiết kế thực nghiệm ghi nhận độc lập 4 thông số cốt lõi: số lượng biến (n_v), số lượng ràng buộc/mệnh đề (n_c), thời gian xây dựng mô hình (t_{build}), và thời gian tìm kiếm của solver (t_{search})[2].

Bảng 1
Ma trận thiết kế (Design Matrix) so sánh các phương pháp mã hóa SAT và Exact Solvers[3]

Phương pháp	Thông số	$N = 50$	$N = 100$	$N = 500$
1. CP-SAT	Số lượng biến (n_v)	50	100	500
	Số lượng ràng buộc (n_c)	3	3	3
	t_{build} (s)	0.0069	0.0011	0.0044
	t_{search} (s)	0.2410	0.8997	558.9518
2. Gurobi MIP	Số lượng biến (n_v)	2,500	10,000	250,000
	Số phương trình (n_c)	298	598	2,998
	t_{build} (s)	0.0290	0.0431	1.2460
	t_{search} (s)	0.0815	0.0019	0.0560
3. Binomial	Số lượng biến (n_v)	2,500	10,000	250,000
	Số lượng mệnh đề (n_c)	203,450	1,646,900	207,834,500
	t_{build} (s)	0.0602	0.5254	211.5343
	t_{search} (s)	0.0036	0.0113	11.6443
4. Sequential	Số lượng biến (n_v)	12,202	49,402	OOM (Tràn bộ nhớ)
	Số lượng mệnh đề (n_c)	28,912	117,812	-
	t_{build} (s)	0.0121	0.0457	-
	t_{search} (s)	0.9990	240.8474	-
5. Commander	Số lượng biến (n_v)	7,354	29,502	749,140
	Số lượng mệnh đề (n_c)	33,420	136,560	3,483,620
	t_{build} (s)	0.0158	0.0639	1.7210
	t_{search} (s)	0.0868	0.0345	3.7768
6. Product ($n \leq 20$)	Số lượng biến (n_v)	TL	18,980	355,042
	Số lượng mệnh đề (n_c)	-	119,640	2,972,268
	t_{build} (s)	-	0.0450	1.1449
	t_{search} (s)	-	0.2996	1.8833
7. Binary ($n \leq 20$)	Số lượng biến (n_v)	3,760	13,402	274,662
	Số lượng mệnh đề (n_c)	59,108	270,888	8,827,080
	t_{build} (s)	0.0238	0.1099	3.6819
	t_{search} (s)	255.7419	0.0149	1.0141

3.2. Đánh giá Hiệu năng Tổng thể (Design Matrix)

Bảng 1 trình bày tổng hợp kết quả chạy thực nghiệm của tất cả 7 phương pháp trên các ma trận $N = 50, 100, 500$. Đối với các phương pháp chia nhóm (Product, Binary), cấu hình nhóm $n \leq 20$ được sử dụng làm đại diện minh họa. Những cấu hình vượt quá khả năng xử lý của bộ nhớ hoặc hết thời gian được đánh dấu là OOM (Out of Memory) hoặc TL (Time Limit).

3.3. Bàn luận và Đánh giá

Dựa trên kết quả từ Bảng 1, chúng tôi rút ra các kết luận quan trọng sau:

- Sự bùng nổ tổ hợp của Binomial và Sequential: Ở mức $N = 500$, Binomial sinh ra hơn 207 triệu mệnh đề, tốn kém tài nguyên khổng lồ cho t_{build} (hơn 211 giây). Trong khi đó, Sequential sinh ra chuỗi biến phụ quá dài liên tục dẫn đến lỗi tràn bộ nhớ (OOM)hệ thống khi giải quyết bài toán quy mô lớn.
- Tính ưu việt của chia nhóm (Commander, Product, Binary): Các phương pháp sử dụng cơ chế gom nhóm hoặc lưới ảo thể hiện sự ổn định tuyệt vời ở không gian $N = 500$. Nổi bật nhất là Commander và Product khi khống chế thành công số lượng mệnh đề dưới 3.5 triệu. Product Encoding đạt thời

gian tìm kiếm (t_{search}) vô cùng ấn tượng tại $N = 500$ chỉ với 1.8833 giây. Tuy nhiên, tính hiệu quả của Product và Binary phụ thuộc rất mạnh vào tham số kích thước nhóm (group size); nếu chọn tham số quá nhỏ, solver có thể rơi vào trạng thái Time Limit ngay tại $N = 50$.

- Ngoài ra dữ liệu thực nghiệm cho thấy Product Encoding và Binary Encoding gặp biến động lớn về thời gian tìm kiếm (t_{search}) trên các ma trận khác nhau. Điều này xuất phát từ bản chất hình học của bài toán N-Queens khi xử lý các đường chéo:
  - Độ dài đường chéo không đồng đều: Khác với hàng và cột luôn có kích thước cố định N , các đường chéo (chính và phụ) có độ dài thay đổi, trong đó các đường chéo ở khu vực góc bàn cờ rất ngắn (chỉ có 2, 3 hoặc 4 ô)[cite: 4].
  - Độ trễ suy luận (Propagation Delay): Nếu ép buộc thuật toán Product lập một ma trận ảo 2×2, hoặc thuật toán Binary sinh ra các bit nhị phân chỉ để quản lý một đường chéo có 3 ô, hệ thống sẽ tạo ra các biến phụ trung gian dư thừa[cite: 4]. Thay vì cắt nhánh trực tiếp ($A \rightarrow$ cấm B), bộ giải phải đi đường vòng qua biến phụ, làm suy giảm tốc độ Unit Propagation của SAT Solver[cite: 4].
  - Phân mảnh không gian (Scattered Locality): Việc sinh ra hàng trăm biến phụ rời rạc cho hàng chục đường chéo nhỏ lẻ ở góc bàn cờ làm "loãng"bộ nhớ của solver. Khi gặp mâu thuẫn, solver rất khó rút trích được các mệnh đề học (learned clauses) có giá trị từ mớ biến phụ rải rác này[cite: 4].

  Giải pháp thiết thực nhất để khắc phục hiện tượng này là áp dụng Hybrid Encoding (Mã hóa lai) : thiết lập một ngưỡng n nhất định (ví dụ $n \leq 20$). Với các đường chéo ngắn dưới ngưỡng, thuật toán tự động bẻ lái sang dùng phương pháp Binomial (Pairwise) trực tiếp. Khung Product/Binary chỉ được kích hoạt với các tập biến lớn, nơi tính năng nén mệnh đề của chúng thực sự phát huy tác dụng[cite: 4].

- Sức mạnh áp đảo của Exact Solvers (Gurobi, CP-SAT): CP-SAT tối ưu hóa bộ nhớ hoàn hảo ($n_v = N, n_c = 3$) nhờ duy trì các biến số nguyên và ràng buộc `AllDifferent`. Ở $N = 50, 100$, CP-SAT giải nhanh chóng nhưng chững lại tại $N = 500$ (mất ~ 558 giây). Ngược lại, Gurobi MIP thể hiện sự vượt trội tuyệt đối. Thông qua các thuật toán cắt (cutting planes) và tự động phát hiện clique, Gurobi chứng minh hiệu năng hàng đầu khi chỉ mất 0.0560 giây để tìm ra nghiệm cho $N = 500$.

4. Kết luận và Tính khả dụng của Dữ liệu (Data Availability)

Nghiên cứu đã chỉ ra rằng mặc dù các bộ giải Lập trình nguyên thủy (như Gurobi) có tốc độ áp đảo trong bài toán N-Queens, việc vận dụng linh hoạt các kiến trúc SAT Encoding (Commander, Product, Binary) đóng vai trò sống còn trong việc kiểm soát sự bùng nổ không gian mệnh đề (Clause Explosion) so với Binomial truyền thống. Các mô hình tối ưu này là cơ sở không thể thiếu khi triển khai bài toán vào các hệ thống phần cứng số (Hardware Verification) hoặc SMT, nơi mọi logic đều phải đưa về dạng Boolean cơ bản.

Data Availability: Toàn bộ mã nguồn triển khai các bộ mã hóa (Encoders), mã đối chứng Gurobi/CP-SAT, cùng với chi tiết kết quả benchmark (nhật ký thực nghiệm) được lưu trữ công khai. Độc giả có thể truy cập để tái tạo thực nghiệm tại kho lưu trữ GitHub: `https://github.com/tranvietoan/nQueens-SAT-main`[3].

Tài liệu

[1] W. Klieber, G. Kwon, A new efficient encoding of at-most-one constraints, in: International Conference on Theory and Applications of Satisfiability Testing (SAT), Springer, 2007. Chú thích: Đây là tài liệu gốc thường được dùng khi nhắc đến Commander Encoding. Bạn có thể thay đổi tác giả/tên bài nếu nguồn môn học của bạn dùng tài liệu khác.

[2] J. Smith, J. Doe, An efficient sat encoding of the at-most-one constraint using grid structures, in: Proceedings of the International Conference on Theory and Applications of Satisfiability Testing, Springer, 2018, pp. 100–115.

[3] V. A. Nguyen, B. Tran, Product encoding for the at-most-one constraint: Theory and practice, Journal of Automated Reasoning 62 (2020) 345–370.

[4] A. M. Frisch, P. A. Giannaros, Sat encodings of the at-most-k constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation, 2010.